Introducción a JavaScript (II)Programación asíncrona y buenas prácticas

Universidad de Sevilla

Departamento de Lenguajes y Sistemas Informáticos

Carlos Arévalo, Daniel Ayala, José Calderón, Margarita Cruz, Inma Hernández, David Ruiz

UNIVERSIDAD DE SEVILLA

Contenidos

- Programación síncrona/asíncrona
  - Introducción
  - Funciones callback
  - Async/await
- Buenas prácticas en JavaScript
  - Consideraciones generales
  - Modo estricto
  - Punto de entrada
  - Uso de módulos
  - Organización del código

Contenidos

Programación síncrona/asíncrona

- **Introducción**
- Funciones callback
- Async/await

Buenas prácticas en JavaScript

- Consideraciones generales
- Modo estricto
- Punto de entrada
- Uso de módulos
- Organización del código

Comunicación síncrona

Cuando hay comunicación síncrona entre dos entes A y B, si A envía un mensaje a B, deja de actuar hasta que B responde

- También llamada “bloqueante”, porque A se bloquea esperando

Comunicación asíncrona

Cuando hay comunicación asíncrona entre dos entes A y B, si A envía un mensaje a B, A puede seguir actuando (incluso enviando otros mensajes a B o otros entes). B responderá cuando pueda y enviará un mensaje de vuelta a A

- También llamada “no bloqueante”

Sincronicidad en JavaScript

JavaScript es **single-threaded** y, en principio, síncrono, es decir, el código se ejecuta en el orden en que está escrito

Sin embargo, también puede tener comportamiento asíncrono

- En estos casos, se definen funciones que gestionarán la recepción de eventos asíncronos, pero no se puede predecir cuándo exactamente se ejecutarán.

Motivos para usar asincronía: eventos que no sabemos si serán respondidos o cuándo, y no pueden quedarse **bloqueando la experiencia de usuario**

- Por ejemplo: consultas a una API

Soluciones

- Funciones callback
- Promesas
- Generadores, async/await…

Casos de uso

En JavaScript, la programación asíncrona se utiliza, principalmente, para:

- Gestionar eventos (**callbacks**): no se puede predecir en qué momento del tiempo se ejecutará el código asociado a un evento, sólo podemos asegurar que será cuando ocurra el evento.
- Operaciones en segundo plano (**async/await**):
  - Evitar que el hilo de ejecución se bloquee si una petición a una API tarda en ser respondida
  - Realizar varias peticiones a una API de manera paralela

Contenidos

Programación síncrona/asíncrona

- Introducción
- **Funciones callback**
- Async/await

Buenas prácticas en JavaScript

- Consideraciones generales
- Modo estricto
- Punto de entrada
- Uso de módulos
- Organización del código

Programación asíncrona: callbacks

Una función callback es cualquier función que se pasa como parámetro para ser ejecutada

- Recordemos que las funciones también son variables:

```
function useCallback(callback) {
  console.log("Running the callback function:");
  callback();
}
function callback1() {
  console.log("Hello");
}
function callback2() {
  console.log("World");
}

useCallback(callback1);
useCallback(callback2);
```

```
Running the callback function:
Hello
Running the callback function:
World
```

Programación asíncrona: callbacks

Generalmente, se usan para definir el código asociado a un evento:

```
function buttonClicked() {
  alert("You pressed the button!");
}

let myButton = document.getElementById("myButton");
myButton.onclick = buttonClicked;
```

No sabemos exactamente **en qué momento** se ejecutará el código de la función de callback, sólo que será cuando **ocurra el evento**.

Contenidos

Programación síncrona/asíncrona

- Introducción
- Funciones callback
- **Async/await**

Buenas prácticas en JavaScript

- Consideraciones generales
- Modo estricto
- Punto de entrada
- Uso de módulos
- Organización del código

Programación síncrona (estándar)

Las llamadas entre funciones normales son bloqueantes: la función que llama queda "en espera"

Flujo de ejecución normal en código síncrono:

```
function myFunction() {
    ...
    ...
    ...
    otherFunction();
    ...
    ...
    ...
}

function otherFunction() {
    ...
    ...
    ...
    ...
    ...
    ...
}
```

Problemas de la programación síncrona en la web

```
function main() {
// When the page loads, load the
// photos in the gallery and the
// trending topics
loadPhotos();
loadTrendingTopics();
}
```

```
function loadPhotos() {
let photos = getPhotosFromAPI();
let gallery = document.
getElementById("gallery");
// Convert the photos from the
// API toHTML, and insert them
// in the page...
```

```
function getPhotosFromAPI() {
// Use a GET request to get
// the photos from a
// RESTful API...
}
```

¿Y si la API tarda 10s en responder?

- La función `getPhotosFromAPI()` queda bloqueada 10s…
- Esto bloquea 10s a `loadPhotos()`
- `main()` también queda bloqueda
- **Esperas en cadena**

Problemas de la programación síncrona en la web

Recordemos: JavaScript es fundamentalmente **single-threaded**. Esto implica que cuando una función queda bloqueada…

- La carga de otros elementos en la página queda también bloqueada esperando
- La gestión de eventos se **detiene**, ya que **todo el hilo queda bloqueado**
- La percepción para el usuario es que la página **deja de responder**

Es, por tanto, **imprescindible** que las operaciones que pueden bloquear durante un período de tiempo perceptible se ejecuten **en segundo plano**.

Solución: `async` / `await`

Una función marcada como `async` no es bloqueante, se ejecuta en paralelo con la función que la invoca

Flujo de ejecución con funciones `async`:

```
async function myFunction() {
    ...
    ...
    ...
    otherFunction();
    ...
    ...
    ...
}

                                  async function otherFunction() {
                                      ...
                                      ...
                                      ...
                                      ...
                                      ...
                                      ...
                                  }
```

Solución anterior: Promesas

Antes, se usaba un tipo de objeto llamado `Promise` para gestionar llamadas asíncronas

- La promesa llamaba a una función de callback ("then") cuando la operación asíncrona finalizaba con éxito, y a otra diferente ("catch") si fallaba

Su uso no es muy intuitivo, y en 2017 se introdujo `async` / `await` para facilitar la programación asíncrona

- Las funciones async son en realidad promesas, pero de forma transparente al programador

```
// Promise-based approach
callPromise()
  .then(callBackIfOk)
  .catch(callBackIfError);

function callBackIfOK(){
  ...
}

function callBackIfError(){
  ...
}
```

Ejemplo: Promesas

```javascript
// Función que devuelve una promesa
function obtenerDatos() {
  return new Promise((resolve, reject) => {
    console.log("Cargando datos...");

    setTimeout(() => {
      const exito = true; // cambia a false para probar

      if (exito) {
        resolve("Datos recibidos correctamente");
      } else {
        reject("Error al obtener los datos");
      }
    }, 2000);
  });
}
// Consumir la promesa
obtenerDatos()
  .then((resultado) => {
    console.log(resultado);
  })
  .catch((error) => {
    console.error(error);
  })
  .finally(() => {
    console.log("Proceso finalizado");
  });
```

1. `new Promise(...)` crea una promesa con dos posibles resultados:
   - resolve → éxito
   - reject → error
2. `setTimeout` simula una operación asíncrona (como una llamada a una API)
3. `.then()` se ejecuta si la promesa se cumple (resolve)
4. `.catch()` se ejecuta si falla (reject)
5. `.finally()` se ejecuta siempre (éxito o error)

Programación asíncrona: `async` / `await`

Las llamadas a servicios externos (por ejemplo, APIs REST) se encapsulan en funciones `async`

- Dado que no sabemos cuándo responderá el servidor (o puede que no responda), deben ejecutarse en segundo plano para no bloquear la ejecución

Sin embargo, las funciones `async` tienen particularidades

- Una función `async` sólo puede ser invocada dentro de otra función `async` (las funciones síncronas no pueden tener llamadas asíncronas en su interior)
- La función que invoca a una función `async` no espera a que ésta termine
  - ¿Cómo usar valores de retorno?

Solución

```
async function main() {
    // When the page loads, load the
    // photos in the gallery and the
    // trending topics
    loadPhotos(); // runs in the background
    loadTrendingTopics(); // runs in parallel
}
```

⬇️

```
async function loadPhotos() {
  let photos = await getPhotosFromAPI();
  let gallery = document.getElementById("gallery");
  // Convert the photos from the API to
  // HTML, and insert them in the page...
}
```

⬇️

```
async function getPhotosFromAPI() {
  // Use a GET request to get the photos from a
  // RESTful API...
}
```

1. Convertimos `getPhotosFromAPI` en asíncrona.
2. Una funcion asíncrona sólo puede ser llamada desde otras funciones asincronas `loadPhotos` y `main` también deben ser asíncrona
3. `loadPhotos` espera a que `getPhotosFromAPI` se ejecute, pero `main` puede seguir con su ejecución en paralelo.

Contenidos

Programación síncrona/asíncrona

- Introducción
- Funciones callback
- Async/await

Buenas prácticas en JavaScript

- **Consideraciones generales**
- Modo estricto
- Punto de entrada
- Uso de módulos
- Organización del código

Consideraciones generales (I)

Usar siempre `;` tras cada instrucción

- No hacerlo puede dar lugar a errores sutiles difíciles de depurar

Usar `let` para declarar variables en lugar de `var`

Nombrar variables y funciones en camelCase

Usar `{}` en lugar de `new Object()` y `[]` en lugar de `new Array()`

Consistencia:

- Usar siempre el mismo tipo de comillas (salvo cuando ocasionalmente sean más útiles otras)
- Usar siempre la misma unidad de indentación (recomendado: 4 espacios o 1 tab)

Cuidado con no confundir `for…of` (para iterar sobre arrays) con `for…in` (para iterar sobre claves de Objects)

Usar siempre el modo estricto

Modo estricto

JavaScript ofrece un modo “estricto” opcional que cambia algunos comportamientos del lenguaje:

- Impide usar variables no declaradas (sin modo estricto se crea automáticamente una global)
- Impide que se pueda cambiar el valor de elementos como NaN y Infinity
- Impide que se pueda definir un Object con propiedades repetidas
- En general, convierte en errores cosas que de otro modo serían advertencias
- Asegura que el código será más compatible con futuras versiones de JavaScript

El modo estricto se activa si aparece "use strict"; al principio del script.

Se recomienda su uso.

Consideraciones generales (y II)

Usar el comparador de igualdad `===` en lugar de `==` (y `!==` para desigualdad en lugar de `!=`)

`==` intenta hacer conversiones de tipo para comprobar igualdad y tiene un comportamiento más errático:

- `8 == "8"` es true
- `"  " == 0` es true
- `[] == ""` es true
- `true == 1` es true
- Todos los anteriores son false con `===`
- `==` no es transitivo:
  - `"0" == 0` y `0 == []` pero `"0" != []`

Función main()

Usar una función de entrada `main()` en los scripts JS asociados a cada vista

- Debe ser `async` si contiene llamadas a una API o cualquier otra operación asíncrona

```
async function main() {
  // ...
}

document.addEventListener("DOMContentLoaded", main);
```

La última línea indica al navegador que debe ejecutar la función `main()` cuando cargue la página

- Por defecto, en JS no existen las funciones de entrada (main), lo estamos simulando

Uso de módulos

Los módulos en JavaScript permiten exportar e importar funciones y variables de otros módulos

Se recomienda su uso ya que, si no, todo lo que sea definido en el ámbito global de un fichero JS será accesible por cualquier otro fichero cargado después

- Esto dificulta la depuración y saber de dónde provienen las variables que se están usando

Para activar el uso de módulos, debemos referenciar nuestros scripts JS indicando `type="module"`

- Cada archivo (script) es un módulo

```
<script type="module" src="js/edit_photo.js"></script>
```

Con `type ="module"` indicamos queel script es un módulo

Uso de módulos: Ejemplo

`script_a.js`

```
"use strict";
let helloWorld = "Hello, world!";
let myObject = {
    name: "John",
    surname: "Doe"
};
let myArray = [1, 2, 3, 4, 5];
export { helloWorld, myObject };
```

`script_b.js`

```
"use strict";
import { helloWorld, myObject }
    from './script_a.js';
console.log(helloWorld);
console.log(myObject);
import { myArray }
    from '/js/script_a.js';
```

- `script_a` y `script_b` están en la **misma carpeta**
- En `script_a`, exportamos sólo lo que queremos que sea accesible por otros módulos mediante `export { … }`
- En `script_b` podemos importar elementos exportados por el primero mediante `import { … } from …`
- En el `from` debemos indicar la ruta del módulo del que queremos importar elementos (absoluta o relativa)
- En `script_b` podemos utilizar los elementos importados

No podemos intentar importar algo que no es exportado por el otro módulo

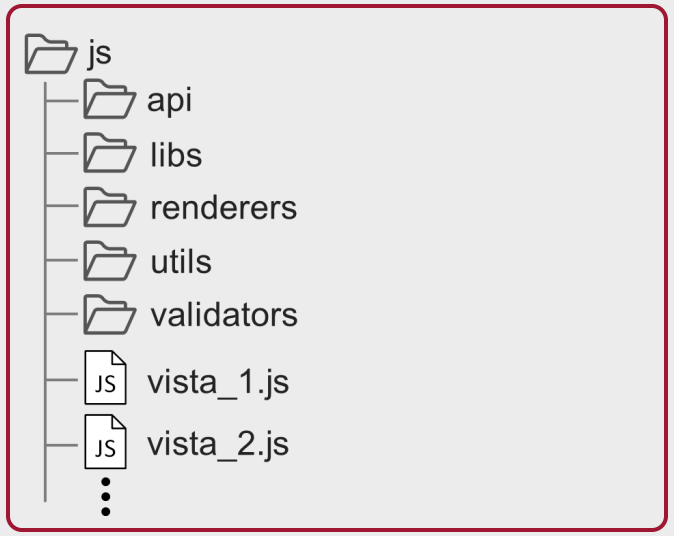
Organización del código

La gestión del código JS de una aplicación web moderna se puede volver compleja si no se siguen pautas para la organización de los archivos

- No existe un estándar claro: depende de la organización y/o del proyecto específico
- Las responsabilidades del código deben estar claramente separadas para facilitar el crecimiento de la aplicación

Organización del código

En IISSI:



- `js/` : contendrá todo lo relacionado con archivos JavaScript.
- `api/` : módulos de comunicación con el API
- `libs/` : librerías externas
- `renderers/` : módulos para transformar objetos JS en HTML
- `utils/` : módulos con funciones de utilidad
- `validators/` : módulos para validar datos de formularios
- `vista_1.js` : módulo por cada vista HTML de la aplicación, se encarga de gestionar eventos del UI.

Conclusiones

La **asincronía** en JavaScript es necesaria para evitar **bloqueos** en la interfaz y mantener una buena **experiencia de usuario**.

Las **callbacks**, las **promesas** y `async/await` son distintas formas de gestionar operaciones que no se completan de manera inmediata.

El uso de `await` permite escribir código asíncrono con un flujo más cercano al código **síncrono**, pero sin bloquear el hilo principal.

Las **buenas prácticas** en JavaScript ayudan a escribir código más **legible**, **mantenible** y menos propenso a errores sutiles.

El uso de **modo estricto**, una función **main()** y **módulos** favorece una estructura más clara y un mejor aislamiento del código.

Una buena **organización del código** separa responsabilidades y facilita que una aplicación web pueda **crecer** sin volverse inmanejable.

Gracias

Universidad de Sevilla

Departamento de Lenguajes y Sistemas Informáticos

UNIVERSIDAD DE SEVILLA